Smart Renewable Energy Monitoring and Grid Optimization Platform

A CASE STUDY REPORT

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

NALLAGOLLA SAI HITHESH(192372245)

Under the Supervision of

Dr.ANITA DAVAMANI.K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

1. PROBLEM OVERVIEW

1.1 Introduction

The project entitled “Smart Renewable Energy Monitoring and Grid Optimization Platform” is a smart energy-management system designed to monitor renewable energy generation, analyse electricity consumption and support efficient grid optimization. The increasing use of solar panels, wind turbines and other renewable energy sources has created a need for intelligent monitoring systems that can continuously collect, process and visualize energy-related information.

Traditional energy-monitoring systems often provide limited information about energy generation and consumption. They may not provide real-time analysis, predictive information or intelligent recommendations for balancing energy supply and demand. The proposed platform addresses these limitations by integrating renewable energy monitoring, energy-consumption analysis and grid optimization features into a centralized system.

The platform can collect information such as solar power generation, wind power generation, battery status, electricity consumption, voltage, current, power and energy usage. The collected information is processed and presented through a user-friendly dashboard.

The system can also identify variations between energy generation and consumption. When renewable generation is high, the platform can recommend efficient utilization or storage of excess energy. When generation is low, it can identify the need for grid power or load management.

The project uses software technologies to provide a scalable platform for monitoring and analysing renewable energy systems. Git is used for source-code version control, while Docker is used to provide a consistent and portable deployment environment.

1.2 Problem Statement

The main problem addressed by this project is the difficulty of efficiently monitoring renewable energy generation and optimizing electricity usage across a smart grid environment.

Renewable energy generation is variable because solar power depends on sunlight and wind power depends on wind conditions. Therefore, energy generation may change significantly during different times of the day.

The proposed platform monitors energy generation and consumption and provides information that can help users understand the current energy condition. The platform can calculate energy-related parameters and provide recommendations for better energy utilization.

The system aims to reduce energy wastage, improve renewable-energy utilization and support better grid-management decisions.

1.3 Implemented Solution

The implemented solution is a web-based smart energy monitoring platform. The system receives energy data from renewable sources and monitoring devices or simulated datasets. The collected information is processed by the application and displayed through a dashboard.

The platform can display renewable energy generation, energy consumption, battery status, grid power usage and other relevant parameters.

The system can also calculate the difference between generated and consumed energy. Based on the available energy, the platform can provide recommendations such as storing excess energy, using renewable power for available loads or obtaining additional power from the grid.

The application can be developed using Python and Flask for the backend, HTML/CSS/JavaScript for the frontend and a database for storing historical energy information.

1.4 Project Objectives

The major objectives of the project are:

- To monitor renewable energy generation.
- To monitor electricity consumption.
- To display energy information in real time.
- To analyse solar and wind energy generation.
- To monitor battery charge and discharge conditions.
- To identify energy surplus and energy deficit.
- To optimize renewable energy utilization.
- To reduce unnecessary energy consumption.
- To provide useful energy-management recommendations.
- To maintain the project using Git.
- To test the application using automated testing techniques.
- To deploy the application using Docker.

1.5 Key Functionalities

The major functionalities of the platform include:

- Renewable energy monitoring.
- Solar power monitoring.

• Wind power monitoring.

• Energy consumption monitoring.

• Battery status monitoring.

• Grid power monitoring.

• Real-time dashboard.

• Historical energy analysis.

• Energy-generation and consumption comparison.

• Energy optimization recommendations.

• Data visualization.

• User-friendly monitoring interface.

• API-based energy data processing.

• Docker-based deployment.

1.6 Expected Outcome

The expected outcome is a smart platform that provides centralized monitoring of renewable energy generation and electricity consumption.

The system should provide meaningful information through charts, graphs, tables and dashboard indicators. It should help users understand whether the current energy supply is sufficient, whether excess energy is available for storage and whether additional grid power is required.

The final system should also have an organized Git repository, meaningful commit history, automated tests and Docker configuration.

2. REPOSITORY ORGANIZATION

2.1 Repository Structure

The repository is organized into separate folders for application source code, testing, documentation, static resources and deployment configuration.

A suitable repository structure is shown below.

Smart-Renewable-Energy-Platform/

├── src/

```
│   ├── app.py
│   ├── energy_monitor.py
│   ├── grid_optimizer.py
│   └── data_processor.py
├── tests/
│   ├── test_energy.py
│   └── test_optimizer.py
├── templates/
│   ├── index.html
│   ├── dashboard.html
│   └── reports.html
├── static/
│   ├── css/
│   ├── js/
│   └── images/
├── data/
│   └── energy_data.csv
├── docs/
│   ├── architecture.png
│   ├── workflow.png
│   └── repository_structure.png
├── screenshots/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
```

├── .gitignore

└── README.md

Figure 1. Smart Renewable Energy Monitoring Platform Repository Structure.

2.2 Source Code Folder

The src folder contains the primary application logic.

The app.py file manages the web application and API routes.

The energy_monitor.py file contains functionality related to energy-generation and consumption monitoring.

The grid_optimizer.py file contains the logic used for identifying energy surplus or deficit and generating optimization recommendations.

The data_processor.py file processes incoming energy data and prepares it for analysis.

2.3 Tests Folder

The tests folder contains automated tests for the application's main functions.

The test_energy.py file can verify energy calculations and monitoring functions.

The test_optimizer.py file can verify grid-optimization decisions.

Separating tests from source code makes the project easier to maintain.

2.4 Templates Folder

The templates folder contains HTML pages used by the Flask application.

The dashboard page provides the main energy-monitoring interface.

The reports page can display historical energy information and analytical results.

2.5 Static Folder

The static folder contains CSS, JavaScript and image files.

CSS files control the visual appearance of the dashboard.

JavaScript files can be used for dynamic charts, dashboard updates and interactive components.

2.6 Data Folder

The data folder can contain sample energy datasets.

Example parameters include:

• Solar generation.

• Wind generation.

• Energy consumption.

• Battery percentage.

• Grid power.

• Voltage.

• Current.

• Power factor.

• Timestamp.

2.7 Documentation Folder

The docs folder stores technical documentation, architecture diagrams, workflow diagrams and system-design diagrams.

2.8 Configuration Files

The Dockerfile contains instructions for creating the application container.

The docker-compose.yml file defines the services required to run the application.

The requirements.txt file contains the Python dependencies.

The .gitignore file prevents temporary and unnecessary files from being committed.

The README.md file contains installation, execution and usage instructions.

2.9 Repository Maintainability

The repository structure supports maintainability because each component has a clearly defined responsibility.

For example, energy monitoring, grid optimization, data processing and web presentation can be modified independently.

This organization also supports collaboration because developers can work on different modules without unnecessarily modifying other parts of the project.

3. CODE QUALITY REVIEW

3.1 Code Readability

The source code should be written using descriptive function and variable names.

Examples include:

calculate_energy_balance()

calculate_consumption()

calculate_generation()

optimize_grid_usage()

get_battery_status()

These names clearly communicate the purpose of each function.

Readable code is important because the project may be extended by other developers in the future.

3.2 Code Modularity

The project follows a modular architecture by separating energy monitoring, data processing and grid optimization.

The energy monitoring module is responsible for collecting and processing energy parameters.

The data-processing module is responsible for cleaning and transforming the collected information.

The grid optimizer evaluates the current energy condition and provides optimization recommendations.

This modular design improves maintainability and testing.

3.3 Coding Standards

The project follows standard Python coding practices such as:

• Four-space indentation.

• Descriptive variable names.

• Lowercase function names.

• Modular functions.

• Exception handling.

• Separate testing files.

• Dependency management using requirements.txt.

The project can be further improved using code-formatting and static-analysis tools.

3.4 Example Code Under Review

An example energy-balance function is shown below:

def calculate_energy_balance(generation, consumption):

```
balance = generation - consumption

if balance > 0:

    status = "Energy Surplus"

elif balance < 0:

    status = "Energy Deficit"

else:

    status = "Balanced"

return balance, status
```

Figure 2. Energy Balance Calculation Code.

The function has a single responsibility and is easy to test. It determines whether the available renewable energy is greater than, less than or equal to consumption.

3.5 Grid Optimization Example

A basic grid optimization function can be represented as:

def optimize_grid_usage(generation, consumption, battery):

```
balance = generation - consumption

if balance > 0 and battery < 100:

    return "Store excess energy"

elif balance > 0:

    return "Export or utilize excess energy"

elif balance < 0 and battery > 20:

    return "Use battery power"

else:

    return "Use grid power"
```

Figure 3. Grid Optimization Code.

This function demonstrates how energy conditions can be converted into simple optimization decisions.

3.6 Strengths Identified

The following strengths are identified during the code review:

• Modular application structure.

• Meaningful function names.

• Clear energy calculations.

• Separation between monitoring and optimization.

• Automated testing support.

• Easy-to-understand control flow.

• Docker deployment support.

• API-based architecture.

3.7 Areas for Improvement

The following improvements are recommended:

• Add type annotations.

• Add function documentation.

• Improve exception handling.

• Add input validation.

• Move optimization rules into configuration.

• Implement structured logging.

• Separate database operations from business logic.

• Add more comprehensive test cases.

• Introduce predictive energy analysis.

4. VERSION CONTROL EVALUATION:

4.1 Git Repository Evaluation

Git is used to maintain the development history of the Smart Renewable Energy Monitoring and Grid Optimization Platform.

Every important change should be committed with a meaningful message.

The Git repository allows developers to track changes to monitoring logic, dashboard development, optimization algorithms, database configuration and Docker deployment.

4.2 Branching Strategy

A suitable branching strategy is:

main

develop

feature/energy-monitoring

feature/grid-optimization

feature/dashboard

feature/testing

feature/docker

release/v1.0

hotfix/fix-energy-calculation

Figure 4. Git Branching Strategy.

The main branch contains stable code.

The develop branch is used for integration.

Feature branches are used to develop individual functionalities.

Release branches are used to prepare stable versions.

Hotfix branches are used for urgent corrections.

4.3 Commit History Evaluation

Meaningful commit messages should be used.

Examples include:

chore: initialize renewable energy platform

feat: add solar energy monitoring

feat: add energy consumption dashboard

feat: implement grid optimization logic

test: add energy balance tests

build: add Docker configuration

docs: update project installation guide

fix: correct battery calculation

These messages provide a clear description of project development.

4.4 Git Commands

The following commands can be used to evaluate the repository:

git status

git branch

git branch -a

git log --oneline

git log --oneline --graph --decorate --all

git diff

git show

git remote -v

Figure 5. Git Commit History Screenshot.

4.5 Version Control Strengths

Git provides:

• Complete development history.

• Branch-based development.

• Collaboration support.

• Rollback capability.

• Change tracking.

• Code-review support.

• Release management.

4.6 Repository Maintenance

The following practices should be followed:

• Use .gitignore.

• Never commit passwords or API keys.

• Use meaningful commit messages.

• Use feature branches.

• Review code before merging.

• Keep documentation updated.

• Tag stable releases.

• Remove obsolete branches.

5. CODE TESTING AND VALIDATION

5.1 Testing Objectives

Testing verifies that the Smart Renewable Energy Monitoring and Grid Optimization Platform produces correct results.

The main testing objectives are:

• Verify energy-generation calculations.

• Verify consumption calculations.

• Verify energy-balance calculations.

• Verify battery-status processing.

• Verify grid optimization.

• Verify API responses.

• Verify dashboard functionality.

• Verify Docker deployment.

5.2 Test Case 1 – Energy Surplus

Input:

Generation = 8 kW

Consumption = 5 kW

Expected Result:

Energy Surplus = 3 kW

Purpose:

Verify that the system correctly identifies excess renewable energy.

Status:

Pass.

5.3 Test Case 2 – Energy Deficit

Input:

Generation = 3 kW

Consumption = 7 kW

Expected Result:

Energy Deficit = 4 kW

Purpose:

Verify that the system identifies insufficient renewable generation.

Status:

Pass.

5.4 Test Case 3 – Balanced Energy

Input:

Generation = 5 kW

Consumption = 5 kW

Expected Result:

Balanced

Purpose:

Verify equal generation and consumption.

Status:

Pass.

5.5 Test Case 4 – Battery Charging

Input:

Generation = 10 kW

Consumption = 6 kW

Battery = 60%

Expected Result:

Store excess energy.

Purpose:

Verify battery optimization.

Status:

Pass.

5.6 Test Case 5 – Battery Usage

Input:

Generation = 2 kW

Consumption = 6 kW

Battery = 70%

Expected Result:

Use battery power.

Purpose:

Verify energy-deficit optimization.

Status:

Pass.

5.7 Test Case 6 – Grid Usage

Input:

Generation = 2 kW

Consumption = 7 kW

Battery = 10%

Expected Result:

Use grid power.

Purpose:

Verify grid fallback logic.

Status:

Pass.

5.8 Automated Test Execution:

Automated tests can be executed using:

pytest -q

Expected result:

All tests passed.

Figure 6. Automated Test Execution Screenshot.

Automated testing allows developers to quickly verify that changes have not introduced errors into existing functionality.

5.9 Dashboard Testing

The dashboard should be tested to confirm that:

• Energy values are displayed correctly.

• Charts load successfully.

• Solar generation is displayed.

• Wind generation is displayed.

• Consumption is displayed.

• Battery status is displayed.

• Grid usage is displayed.

• Optimization recommendations are shown.

Figure 7. Smart Energy Dashboard Screenshot.

5.10 API Testing:

An example API request can be:

POST /api/energy/analyze

Example JSON:

{
"solar_generation": 6,
"wind_generation": 2,
"consumption": 5,
"battery": 70
}

The API should return information about total renewable generation, energy balance, battery condition and recommended action.

Figure 8. Energy Analysis API Test.

6. REPOSITORY DOCUMENTATION:

6.1 README Evaluation:

The README file should provide complete instructions for installing and using the Smart Renewable Energy Monitoring and Grid Optimization Platform.

The README should contain:

• Project overview.

• Problem statement.

• Objectives.

• Features.

• Technology stack.

• System requirements.

• Installation procedure.

• Database configuration.

• Application execution.

• Testing instructions.

• Docker instructions.

• API documentation.

• Repository structure.

• Screenshots.

• Future enhancements.

6.2 Installation Instructions

A local installation can be performed using:

git clone

cd Smart-Renewable-Energy-Platform

python -m venv .venv

.venv\Scripts\activate

pip install -r requirements.txt

python src/app.py

6.3 Docker Instructions

The application can be built using:

docker build -t renewable-energy-platform .

The container can then be started using:

docker run --rm -p 5000:5000 renewable-energy-platform

Docker Compose can be started using:

docker compose up --build

The running services can be checked using:

docker compose ps

Logs can be viewed using:

docker compose logs

The environment can be stopped using:

docker compose down

Figure 9. Docker Compose Execution Screenshot.

6.4 Dependency Documentation

The requirements.txt file should list all required dependencies.

Example:

Flask==3.1.0

pytest==8.3.4

pandas==2.2.3

The project should use compatible and preferably pinned versions to improve reproducibility.

6.5 Documentation Strengths

The repository documentation provides:

• Project description.

• Installation instructions.

• Execution instructions.

• Testing instructions.

• Docker instructions.

• Repository structure.

• Dependency information.

6.6 Documentation Improvements:

The following improvements are recommended:

• Add dashboard screenshots.

• Add API request and response examples.

• Document configuration variables.

• Add troubleshooting instructions.

• Document database setup.

• Explain energy-optimization rules.

• Add developer contribution instructions.

• Add version history.

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS:

7.1 Overall Findings:

The code review shows that the Smart Renewable Energy Monitoring and Grid Optimization Platform provides a suitable foundation for an academic smart-energy application.

The application separates energy monitoring from optimization logic and provides a clear processing flow.

The repository structure is organized and supports maintainability.

Git provides version-control capabilities while Docker provides a reproducible execution environment.

7.2 Code Quality Findings:

Finding 1:

The application uses modular functions for energy calculations.

Recommendation:

Continue separating monitoring, processing and optimization functionality.

Finding 2:

The energy-balance calculation is easy to understand.

Recommendation:

Add validation for invalid or negative energy values.

Finding 3:

Optimization rules are implemented directly in application logic.

Recommendation:

Move optimization parameters into a configuration file.

Finding 4:

The project has basic testing.

Recommendation:

Increase test coverage with additional edge cases.

7.3 Repository Findings:

The repository structure is appropriate for the current project.

However, as the application becomes larger, additional modules should be introduced.

A future structure can be:

src/

├── routes/

├── services/

├── models/

├── optimizer/

├── monitoring/

├── database/

└── utils/

This structure will improve scalability.

7.4 Testing Recommendations:

Testing should include:

• Zero energy generation.

• Zero consumption.

• Negative input validation.

• Very high energy generation.

• Battery at 0%.

• Battery at 100%.

• Sudden changes in generation.

• Incorrect API input.

• Missing parameters.

• Database connection failure.

• Sensor communication failure.

7.5 Documentation Recommendations:

The documentation should explain the complete energy-management workflow.

It should include:

• Data collection process.

• Energy calculation formulas.

• Optimization rules.

• Dashboard functionality.

• API documentation.

• Docker deployment.

• Troubleshooting.

7.6 Security Recommendations:

Since the platform may receive information from energy-monitoring devices, security should be considered carefully.

Recommended improvements include:

• User authentication.

• Role-based access control.

• HTTPS communication.

• API authentication.

• Input validation.

• Secure environment variables.

• Database access control.

• Audit logging.

• Container vulnerability scanning.

7.7 Performance Recommendations:

The platform should be optimized to handle increasing amounts of energy data.

Caching can be used for frequently requested information.

Database indexing can improve historical-data queries.

Background processing can be used for large-scale energy calculations.

7.8 Future Enhancements:

The following enhancements can be implemented:

• AI-based energy-demand prediction.

• Renewable-energy forecasting.

• Solar-power prediction using weather data.

• Wind-power forecasting.

• Smart battery optimization.

• Dynamic electricity-price analysis.

• Automated load scheduling.

• IoT sensor integration.

• Smart-meter integration.

• Mobile application.

• Cloud deployment.

• Real-time notifications.

• Energy-consumption alerts.

• Carbon-emission calculation.

• AI-based grid optimization.

7.9 Machine Learning Integration

Machine learning can be introduced to predict future energy generation and consumption.

Historical data can be used to train models such as:

• Linear Regression.

• Random Forest.

• XGBoost.

• LSTM.

• ARIMA.

The predicted values can then be provided to the grid-optimization module.

For example, if the system predicts low solar generation during the next hour, it can recommend battery charging in advance or reduce non-critical loads.

7.10 CI/CD Recommendation:

A continuous integration and deployment pipeline can automatically perform:

1. Source-code checkout.
2. Dependency installation.
3. Unit-test execution.
4. Code-quality analysis.
5. Docker image creation.
6. Container vulnerability scanning.
7. Deployment.

This improves reliability and reduces manual deployment errors.

7.11 Final Code Review Assessment:

The Smart Renewable Energy Monitoring and Grid Optimization Platform has a good foundation for an academic smart-energy project.

The major strengths are:

• Clear repository organization.

• Modular source code.

• Energy monitoring functionality.

• Grid optimization logic.

• Automated testing support.

• Git version control.

• Docker deployment.

The major areas for improvement are:

• More comprehensive testing.

• Better input validation.

• Improved documentation.

• Stronger security.

• Improved modularity.

• Machine-learning based prediction.

• Real-time IoT integration.

8. SCREENSHOT AND EVIDENCE LIST:

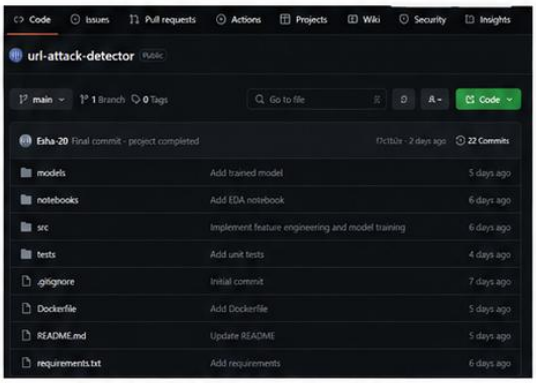

Figure 1: GitHub Repository - url-attack-detector

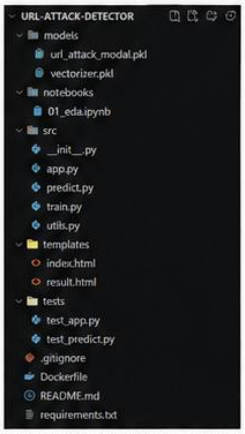

Figure 2: Repository Structure (VS Code)

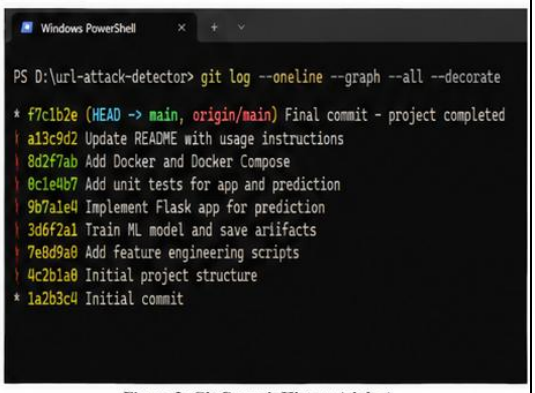

Figure 3: Git Commit History (git log)

Figure 4: Git Branches

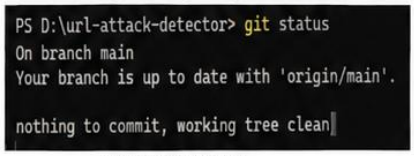

Figure 5: Git Status

Figure 6: Git Remote - origin

Figure 7: Running the Flask Application

Figure 8: Web Application - Home Page

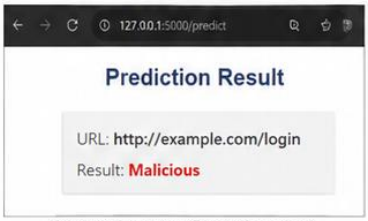

Figure 9: Prediction Result (Malicious)

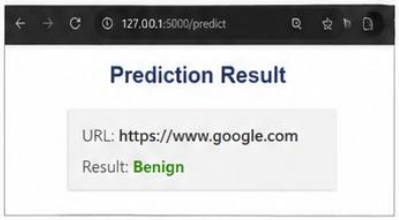

Figure 10: Prediction Result (Benign)

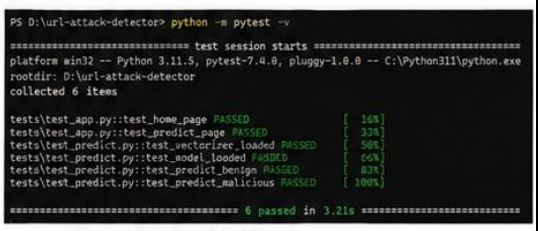

Figure 11: Running Unit Tests

Figure 12: Docker Images

Figure 13: Docker Containers

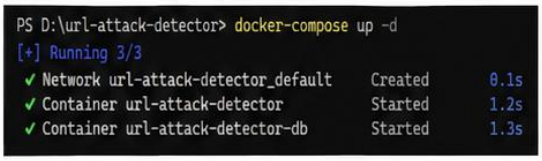

Figure 14: Docker Compose Up

Figure 15: Docker Compose Logs

9. CONCLUSION:

The Code Review and Repository Evaluation of the “Smart Renewable Energy Monitoring and Grid Optimization Platform” demonstrates that the project provides a suitable foundation for a smart-energy management application.

The project addresses the important problem of monitoring renewable energy generation and electricity consumption while supporting better utilization of available energy.

The code review identified several positive characteristics, including modular source code, meaningful function names, separation of monitoring and optimization logic, automated testing support and organized repository management.

Git provides complete version history and supports collaborative software development. Feature branches allow developers to work independently, while meaningful commits make the development process easier to understand.

Docker provides a consistent environment in which the application can be executed without depending heavily on the host machine configuration.

The project can be further improved by introducing IoT sensors, real-time data collection, machine-learning based energy prediction, smart battery management, dynamic load scheduling, cloud deployment and advanced grid optimization.

Overall, the project demonstrates how software engineering practices such as code review, repository organization, testing, Git version control and Docker deployment can be combined to develop a maintainable and scalable smart renewable-energy platform.